



Học viện Công nghệ Bưu chính Viễn thông
Khoa Công nghệ thông tin 1

Phân tích và khai phá dữ liệu văn bản

Chương 3: Phân loại văn bản

TS. Nguyễn Thị Mai Trang
maitrangnguyen.pt@gmail.com

- Giới thiệu phân loại văn bản
- Trích chọn đặc trưng
 - Túi từ (Bag of Words)
 - TF-IDF
 - Véc-tơ từ (Word Embeddings)
- Các thuật toán phân loại
 - Bayes đơn giản (Naive Bayes)
 - Máy hỗ trợ véc-tơ (SVM - Support Vector Machine)
- Đánh giá các mô hình phân loại
- Bài tập lớn

Giới thiệu phân loại văn bản

- Phân loại văn bản (Text Classification) là quá trình gán các nhãn (categories/classes) cho các tài liệu văn bản dựa trên nội dung của chúng.
- **Mục tiêu:** Tự động hóa việc tổ chức, sắp xếp, và hiểu dữ liệu văn bản phi cấu trúc.
- **Ví dụ:**
 - Phân loại email spam/non-spam.
 - Phân loại tin tức theo chủ đề (thể thao, chính trị, kinh tế...).
 - Phân loại bình luận sản phẩm (tích cực/tiêu cực).
 - Phân loại bài báo khoa học theo lĩnh vực.

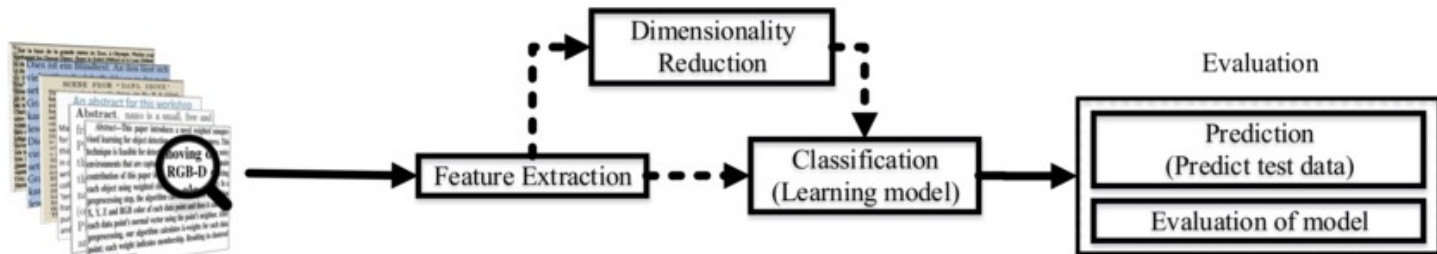


Tại sao cần phân loại văn bản?

- **Quản lý thông tin:** Giúp tổ chức lượng lớn dữ liệu văn bản hiệu quả.
- **Tự động hóa:** Giảm thiểu công sức thủ công, tăng tốc độ xử lý.
- **Tìm kiếm và truy xuất thông tin:** Cải thiện độ chính xác khi tìm kiếm.
- **Phân tích dữ liệu:** Hỗ trợ các quyết định kinh doanh, nghiên cứu.
- **Cá nhân hóa:** Đề xuất nội dung phù hợp với người dùng.



Quy trình tổng quan



Quy trình phân loại văn bản gồm các bước cơ bản như sau:

- **Tiền xử lý dữ liệu (Preprocessing):** Làm sạch và chuẩn hóa văn bản.
- **Trích chọn đặc trưng (Feature Extraction):** Chuyển đổi văn bản thành dạng số học.
- **Xây dựng mô hình (Model Building):** Huấn luyện thuật toán phân loại trên dữ liệu đã được trích chọn đặc trưng.
- **Đánh giá mô hình (Model Evaluation):** Đánh giá hiệu suất của mô hình.

- Loại bỏ ký tự đặc biệt, số
- Chuyển về chữ thường
- Tách từ (Tokenization)
- Loại bỏ stop words
- POS tagging
- NER
- Stemming/Lemmatization

Trích chọn đặc trưng

- **Trích chọn đặc trưng** là quá trình biến đổi dữ liệu văn bản (phi cấu trúc) thành các biểu diễn số học (vector, ma trận) mà các thuật toán học máy có thể hiểu và xử lý được.
- **Tầm quan trọng:** Chất lượng của đặc trưng ảnh hưởng trực tiếp đến hiệu suất của mô hình phân loại.
- **Các phương pháp phổ biến:**
 - Túi từ (Bag of Words - BoW)
 - TF-IDF
 - Véc-tơ từ (Word Embeddings)

Túi từ (Bag of Words - BoW)

- **Bag of Words (BoW)** là một kỹ thuật biểu diễn văn bản (chứ không chỉ là trích xuất tính năng) được sử dụng trong xử lý ngôn ngữ tự nhiên (NLP) và học máy. Nó biểu diễn một đoạn văn bản (như một câu, một đoạn, hoặc một tài liệu) như một tập hợp các từ và sự xuất hiện của chúng.
- **Ý tưởng:** Biểu diễn văn bản như một "túi" các từ, bỏ qua ngữ pháp và thứ tự từ, chỉ quan tâm đến tần suất xuất hiện của mỗi từ.
- **Cách hoạt động:**
 - Tạo một từ điển (vocabulary) chứa tất cả các từ duy nhất từ toàn bộ tập dữ liệu.
 - Mỗi văn bản được biểu diễn dưới dạng một véc-tơ có kích thước bằng kích thước từ điển.
 - Giá trị tại mỗi vị trí trong véc-tơ là số lần xuất hiện của từ tương ứng trong từ điển.

Túi từ (Bag of Words - BoW)

Document D1	<i>The child makes the dog happy</i> the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	<i>The dog makes the child happy</i> the: 2, child: 1, makes: 1, dog: 1, happy: 1



	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

Túi từ (Bag of Words - BoW)

- Văn bản 1: "Tôi thích món phở Việt Nam."
- Văn bản 2: "Phở là món ăn ngon của Việt Nam."
- Bước 1: Xây dựng từ điển (Vocabulary):
 - {"Tôi": 0, "thích": 1, "món": 2, "phở": 3, "Việt Nam": 4, "là": 5, "ăn": 6, "ngon": 7, "của": 8}
- Bước 2: Biểu diễn véc-tơ:
 - Văn bản 1: [1, 1, 1, 1, 1, 0, 0, 0, 0] (Tôi:1, thích:1, món:1, phở:1, Việt Nam:1)
 - Văn bản 2: [0, 0, 1, 1, 1, 1, 1, 1, 1] (món:1, phở:1, Việt Nam:1, là:1, ăn:1, ngon:1, của:1)

Túi từ (Bag of Words - BoW)

- **Ưu điểm:**
 - Đơn giản, dễ hiểu và dễ triển khai.
 - Hiệu quả với các tập dữ liệu lớn.
- **Nhược điểm:**
 - **Mất thông tin ngữ cảnh:** Không quan tâm đến thứ tự từ, dẫn đến mất ý nghĩa ngữ pháp.
 - **Vấn đề từ hiếm (Sparsity):** Từ điển rất lớn, nhiều từ chỉ xuất hiện một vài lần, tạo ra các véc-tơ thưa.
 - **Vấn đề từ mới (Out-of-Vocabulary - OOV):** Không xử lý được từ mới chưa có trong từ điển.
 - **Không nắm bắt được mối quan hệ ngữ nghĩa:** "tốt" và "xuất sắc" có ý nghĩa tương đồng nhưng được coi là độc lập.

TF-IDF

- **TF-IDF** (*Term Frequency-Inverse Document Frequency*) là một phương pháp thống kê nhằm đánh giá mức độ quan trọng của một từ đối với một tài liệu trong một tập hợp tài liệu.
- **Ý tưởng:** Một từ quan trọng là từ xuất hiện nhiều trong một tài liệu cụ thể (TF cao) nhưng lại ít xuất hiện trong các tài liệu khác (IDF cao).

TF-IDF: công thức

$$TF(t, d) = \frac{(\text{Number of occurrences of term } t \text{ in document } d)}{(\text{Total number of terms in the document } d)}$$

$$IDF(t, D) = \log_e \frac{(\text{Total number of documents in the corpus})}{(\text{Number of documents with term } t \text{ in them})}$$

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

- TF là tần suất xuất hiện của một từ t trong tài liệu d .
- IDF là Mức độ hiếm của một từ trong toàn bộ tập dữ liệu. Từ càng hiếm, IDF càng cao.

TF-IDF: công thức

- Công thức:
$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t,D)$$
- **Ý nghĩa:** Từ có giá trị TF-IDF cao là từ quan trọng đối với tài liệu đó và hiếm trong tập dữ liệu.
- **Ví dụ:**
 - D1: "AI is the future."
 - D2: "AI and machine learning."
 - Tổng số tài liệu $N = 2$.
 - Hãy tính TF-IDF của từ **machine** trong D1 và D2?

TF-IDF: ví dụ 1

Bước 1: DF và IDF

- $df(machine) = 1$ (chỉ xuất hiện trong D2)
- Tổng số tài liệu $N = 2$

$$idf(machine) = \log_{10}\left(\frac{N}{df}\right) = \log_{10}(2) = 0.30103$$

Bước 2: TF

- $TF(machine, D1) = 0/4 = 0$
- $TF(machine, D2) = 1/4 = 0.25$

Bước 3: TF-IDF

- $TF-IDF(machine, D1) = 0 \times 0.30103 = 0$
- $TF-IDF(machine, D2) = 0.25 \times 0.30103 \approx 0.07526$

Kết luận: từ **machine** có trọng số TF-IDF ≈ 0.0753 trong **D2**, và **0** trong **D1**.

Làm mịn IDF

$$\text{IDF}(t) = \log \left(\frac{N + 1}{\text{df}(t) + 1} \right) + 1$$

Trong đó

N : tổng số văn bản trong corpus.

$\text{df}(t)$: số văn bản chứa từ t .

(công thức làm mịn theo sklearn)

- **Việc cộng thêm 1 vào cả tử số và mẫu số** giúp tránh trường hợp mẫu số bằng 0 nếu một từ chưa bao giờ xuất hiện trong tập huấn luyện (training set). Đây là một kỹ thuật làm mịn (smoothing) để đảm bảo mọi từ đều có giá trị IDF hợp lệ, kể cả những từ không có trong tập dữ liệu.
- **Việc cộng thêm 1 vào toàn bộ công thức** ($\log(\dots)+1$) giúp tránh trường hợp IDF bằng 0 khi một từ xuất hiện trong tất cả các văn bản, làm cho TF-IDF của từ đó không bị triệt tiêu hoàn toàn.

TF-IDF: ví dụ 2

D1: "data science is fun"

D2: "big data and machine learning"

D3: "I love learning science"

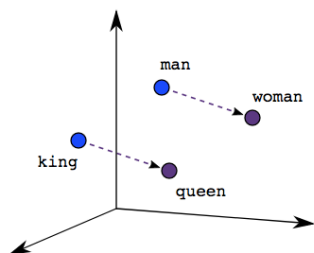
- a) Hãy tính TF-IDF của tất cả các từ trong D1, D2, D3?
- b) Hãy tính TF-IDF(machine) trong D1, D2, D3 sử dụng công thức làm mịn IDF?

TF-IDF: Ưu nhược điểm

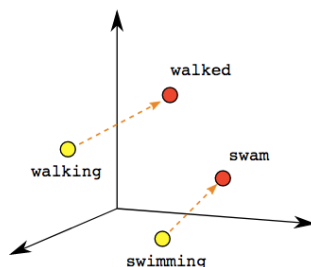
- **Ưu điểm:**
 - Phản ánh tốt hơn mức độ quan trọng của từ so với BoW.
 - Giảm ảnh hưởng của các từ phổ biến.
 - Dễ hiểu và dễ tính toán.
- **Nhược điểm:**
 - Vẫn là biểu diễn thưa (sparse representation).
 - Không nắm bắt được mối quan hệ ngữ nghĩa giữa các từ (tương tự BoW).
 - Không xử lý tốt từ đồng nghĩa, đa nghĩa.
 - Không quan tâm đến thứ tự từ.

Véc-tơ từ (Word Embeddings)

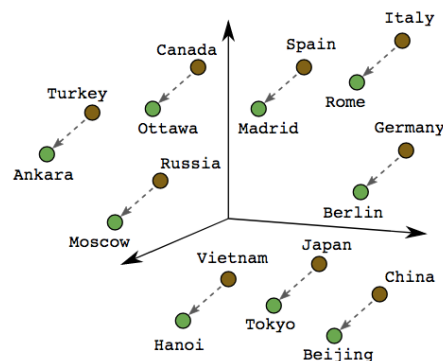
- **Word Embeddings** biểu diễn từ dưới dạng các véc-tơ số thực dày đặc (dense vectors) trong không gian nhiều chiều.
- **Mục tiêu:** Nắm bắt được mối quan hệ ngữ nghĩa và ngữ pháp giữa các từ. Các từ có ý nghĩa tương đồng sẽ có véc-tơ gần nhau trong không gian.



Male-Female



Verb Tense



Country-Capital

Véc-tơ từ (Word Embeddings)

Khắc phục nhược điểm của BoW/TF-IDF:

- **Xử lý ngữ nghĩa:** Các từ có ý nghĩa tương tự sẽ có véc-tơ gần nhau, giúp mô hình hiểu được mối quan hệ.
- **Biểu diễn dày đặc:** Giảm kích thước véc-tơ, hiệu quả hơn cho các mô hình học sâu.
- **Giảm chiều:** Thay vì véc-tơ hàng trăm nghìn chiều (BoW), Word Embeddings thường chỉ vài trăm chiều.
- **Xử lý OOV tốt hơn:** Có thể ước lượng véc-tơ cho từ mới dựa trên ngữ cảnh hoặc các thành phần của từ.

Các mô hình WE phổ biến

- **Word2Vec:**
 - **Skip-gram:** Dự đoán các từ ngữ cảnh xung quanh một từ cho trước.
 - **CBOW (Continuous Bag of Words):** Dự đoán từ hiện tại dựa trên các từ ngữ cảnh.
- **GloVe (Global Vectors for Word Representation):** Kết hợp cả phương pháp cục bộ (từ ngữ cảnh) và toàn cục (ma trận đồng xuất hiện).
- **FastText:** Mở rộng Word2Vec bằng cách tính toán các véc-tơ cho các từ con (subword units), giúp xử lý tốt các từ hiếm và từ mới.
- **Các mô hình dựa trên Transformer (BERT, GPT, ...):** Tạo ra các véc-tơ ngữ cảnh động, tức là cùng một từ có thể có các véc-tơ khác nhau tùy theo ngữ cảnh mà nó xuất hiện.

Word2Vec

Word2Vec là một mô hình học biểu diễn từ vụng (word embeddings). Word2Vec học cách gán cho mỗi từ một vector ngắn, đặc, nằm trong một **không gian liên tục**, sao cho:

- Từ tương đồng ngữ nghĩa sẽ nằm gần nhau.
- Quan hệ ngữ nghĩa được bảo toàn

Word2Vec: Hai mô hình chính

- **CBOW**

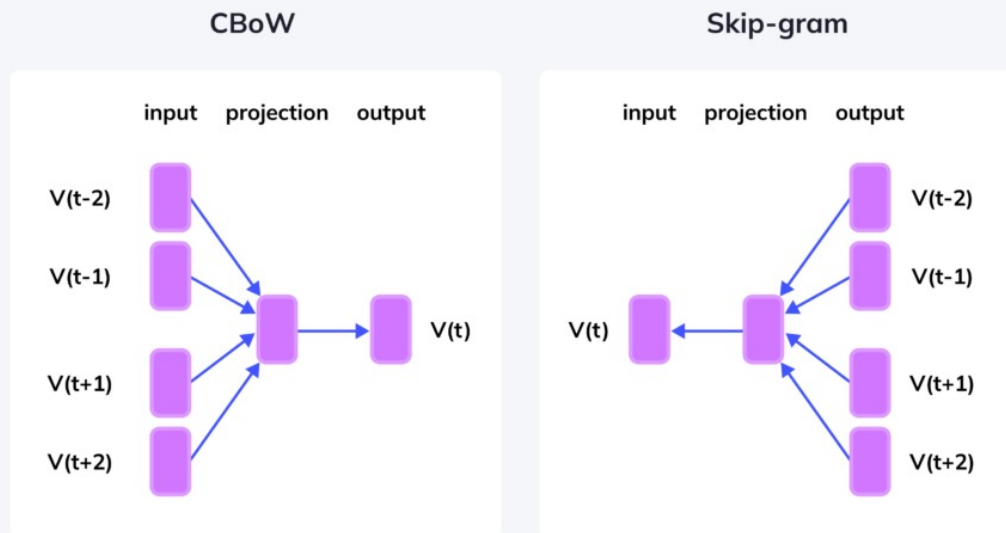
Input: từ xung quanh (context).

Output: từ trung tâm.

- **Skip-gram**

Input: từ trung tâm.

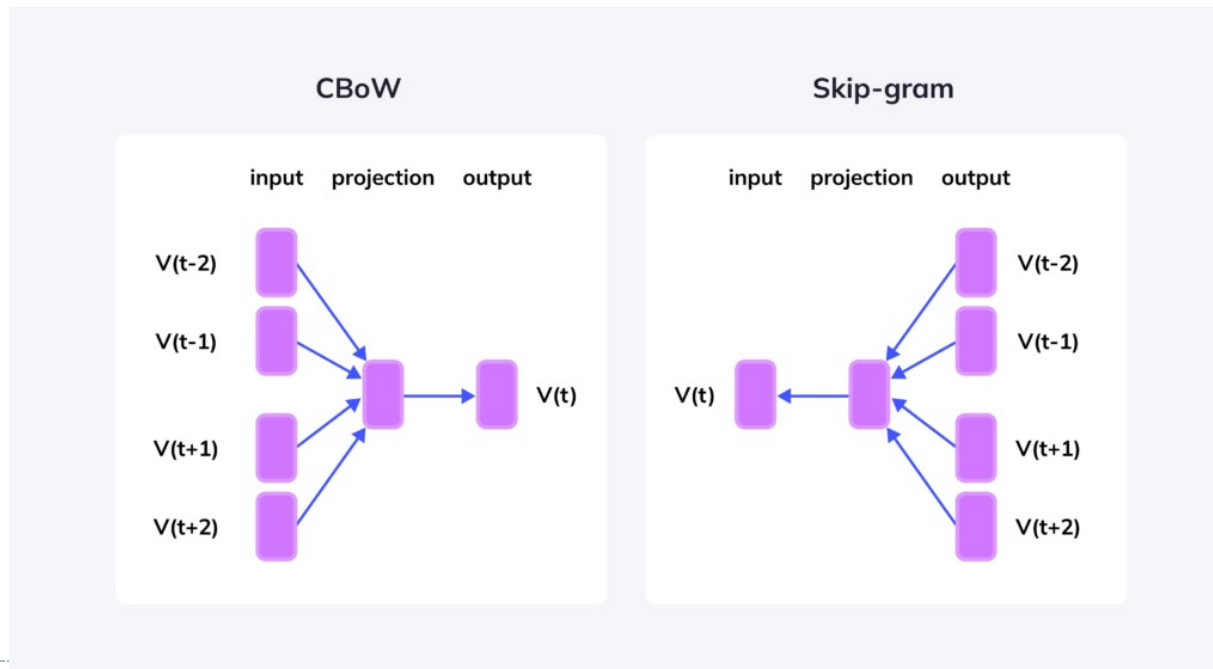
Output: từ xung quanh.



Word2Vec: Hai mô hình chính

Câu: “*The cat sat on the mat*”

- CBOW: dự đoán “sat” từ (“The”, “cat”, “on”, “the”, “mat”).
- Skip-gram: dự đoán (“The”, “cat”, “on”, “the”, “mat”) từ “sat”.



Word2Vec: Cơ chế học và ứng dụng

- **Học vector:**
 - Khởi tạo ngẫu nhiên, huấn luyện qua tối ưu hàm mất mát (softmax / negative sampling).
 - Vector sau huấn luyện chứa thông tin ngữ nghĩa.
- **Tính chất:**
 - Distributed representation: mỗi chiều không có ý nghĩa riêng.
 - Semantic similarity: từ đồng nghĩa có cosine similarity cao.
 - Vector arithmetic:
 - $\text{Paris} - \text{France} + \text{Italy} \approx \text{Rome}$
 - $\text{Walking} - \text{Walk} + \text{Swim} \approx \text{Swimming}$
- **Ứng dụng:**
 - Tìm từ đồng nghĩa / gần nghĩa
 - Clustering & Classification văn bản
 - Khởi tạo embedding cho RNN / Transformer

WB: Ưu nhược điểm

- **Ưu điểm:**

- Nắm bắt được mối quan hệ ngữ nghĩa và ngữ pháp.
- Biểu diễn dày đặc, hiệu quả cho học máy và học sâu.
- Xử lý tốt hơn các từ đồng nghĩa, đa nghĩa (với các mô hình ngữ cảnh).

- **Nhược điểm:**

- Tốn tài nguyên tính toán để huấn luyện các mô hình lớn.
- Hiệu quả phụ thuộc vào chất lượng và kích thước của tập dữ liệu huấn luyện.
- Các mô hình Word2Vec/GloVe vẫn tạo ra véc-tơ tĩnh (một từ luôn có cùng một véc-tơ bất kể ngữ cảnh).

Word2Vec: One-hot vector

- Mỗi từ trong từ vựng được mã hóa thành một vector nhị phân.
- Chỉ một phần tử bằng **1**, còn lại bằng **0**.

Ví dụ:

vocab = {*tôi*, *thích*, *học*, *AI*}:

tôi → [1, 0, 0, 0]

thích → [0, 1, 0, 0]

học → [0, 0, 1, 0]

AI → [0, 0, 0, 1]

Word2Vec: Hạn chế của one-hot

- Vector **quá dài** khi vocab lớn.
- **Thưa thớt** (sparse) $\rightarrow$ hầu hết là số 0.
- Không biểu diễn **nghĩa**: "tôi" và "chúng tôi" xa nhau hoàn toàn.

Word2Vec: sử dụng One-hot trong W2V

- Input: one-hot vector.
- Nhân với ma trận trọng số $W \rightarrow$ vector nhúng (embedding).
- Embedding này có số chiều nhỏ hơn nhiều, ví dụ 100 thay vì 100.000.

Word2Vec: sử dụng One-hot trong W2V

Giả sử vocab = 4, embedding size = 2.

$$W = \begin{bmatrix} 0.2 & 0.4 \\ 0.1 & 0.3 \\ 0.7 & 0.8 \\ 0.5 & 0.6 \end{bmatrix}$$

$$\text{"tôi"} = [1, 0, 0, 0] \times W = [0.2, 0.4]$$

$$\text{"thích"} = [0, 1, 0, 0] \times W = [0.1, 0.3]$$

Từ embedding đến softmax

- Vector nhúng tiếp tục nhân với ma trận W' (2×4).
- Ra vector logit (4 chiều).
- Áp dụng softmax $\rightarrow$ xác suất dự đoán context word.

Word2Vec: Dự đoán từ

Trung tâm (center) = "tôi"

- $\text{Embedding}(\text{"tôi"}) = [0.2, 0.4]$
- Nhân $W' \rightarrow \text{logit} = [1.2, 2.3, 0.7, 1.1]$
- Softmax $\rightarrow [0.1, 0.6, 0.2, 0.1]$

=> Mô hình dự đoán từ context = "thích" (xác suất cao nhất).

Học như thế nào?

- Mô hình tính loss = $-\log P(\text{context} | \text{center})$.
- Dùng backpropagation để cập nhật W và W' .
- Sau nhiều lần lặp:
 - Từ xuất hiện gần nhau $\rightarrow$ embedding gần nhau.

Kết quả cuối cùng

- "tôi" gần "thích".
 - "học" gần "AI".
 - Các vector embedding nén thông tin ngữ nghĩa.
- Đây là cách Word2Vec biến **one-hot thưa thớt** thành **embedding giàu ngữ nghĩa**.

Vector từ trung bình có trọng số TF-IDF

- **TF-IDF weighted average word embeddings** là kỹ thuật kết hợp sức mạnh của cả TF-IDF và Word2Vec.
- Thay vì chỉ đơn giản là tính trung bình các vector từ, chúng ta gán trọng số cho vector của mỗi từ bằng điểm TF-IDF của nó. Điều này giúp tăng tầm quan trọng cho các từ liên quan nhất trong một tài liệu.

$$TWA(D) = \frac{\sum_{w=1}^n wv(w) \times tfidf(w)}{n}$$

Trong đó **$TWA(D)$** là vector từ trung bình có trọng số TF-IDF đại diện cho tài liệu D, các từ $w_1, w_2, \dots, w_n$, **$wv(w)$** là vector đại diện từ và **$tfidf(w)$** là trọng số TF-IDF cho từ w

Vector từ trung bình có trọng số TF-IDF

- Pseudo-code cho thuật toán tính vector từ trung bình có trọng số TF-IDF

```
model := the word2vec model we built
vocabulary := unique_words(model)
document := [words]
tfidfs := [tfidf(word) for each word in words]
matched_word_wts := 0
vector := []

for word in words:
    if word in vocabulary:
        word_vector := model[word]
        weighted_word_vector := tfidfs[word] x word_vector
        vector := vector + weighted_word_vector
        matched_word_wts := matched_word_wts + tfidfs[word]

tfidf_wtd_avgd_word_vector := vector / matched_word_wts
```

Các thuật toán phân loại

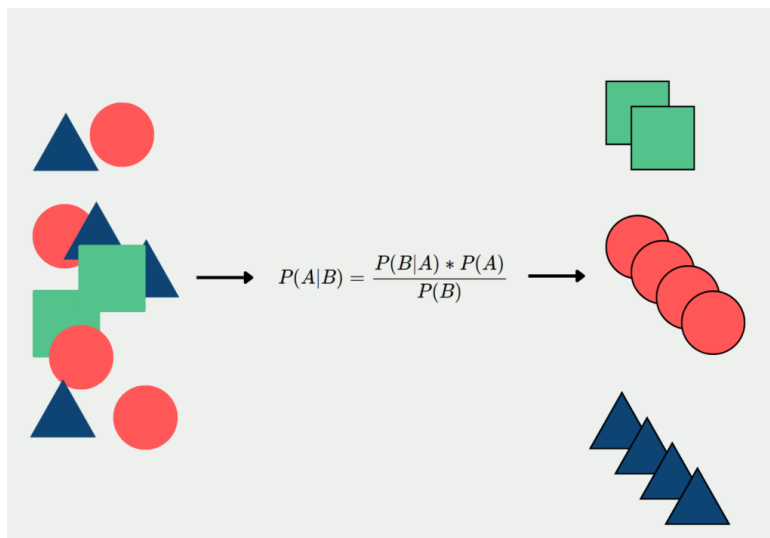
- Sau khi văn bản được chuyển đổi thành các véc-tơ số, chúng ta cần một thuật toán học máy để học cách ánh xạ các véc-tơ này tới các nhãn phân loại.
- **Các thuật toán phổ biến:**
 - Bayes đơn giản (Naive Bayes)
 - Máy hỗ trợ véc-tơ (Support Vector Machine - SVM)
 - Hồi quy Logistic
 - K-Nearest Neighbors (K-NN)
 - Cây quyết định (Decision Trees)
 - Rừng ngẫu nhiên (Random Forest)
 - Mạng nơ-ron (Neural Networks) - Học sâu

Các thuật toán phân loại

- Bayes đơn giản
- SVM

Bayes đơn giản (Naive Bayes)

- **Naive Bayes** là một thuật toán phân loại dựa trên **Định lý Bayes**, với giả định "ngây thơ" (naive) rằng các đặc trưng là độc lập với nhau.



Bayes đơn giản (Naive Bayes)

- **Naive Bayes** là một thuật toán phân loại dựa trên **Định lý Bayes**, với giả định "ngây thơ" (naive) rằng các đặc trưng là độc lập với nhau.
- Công thức cơ bản của Định lý Bayes:

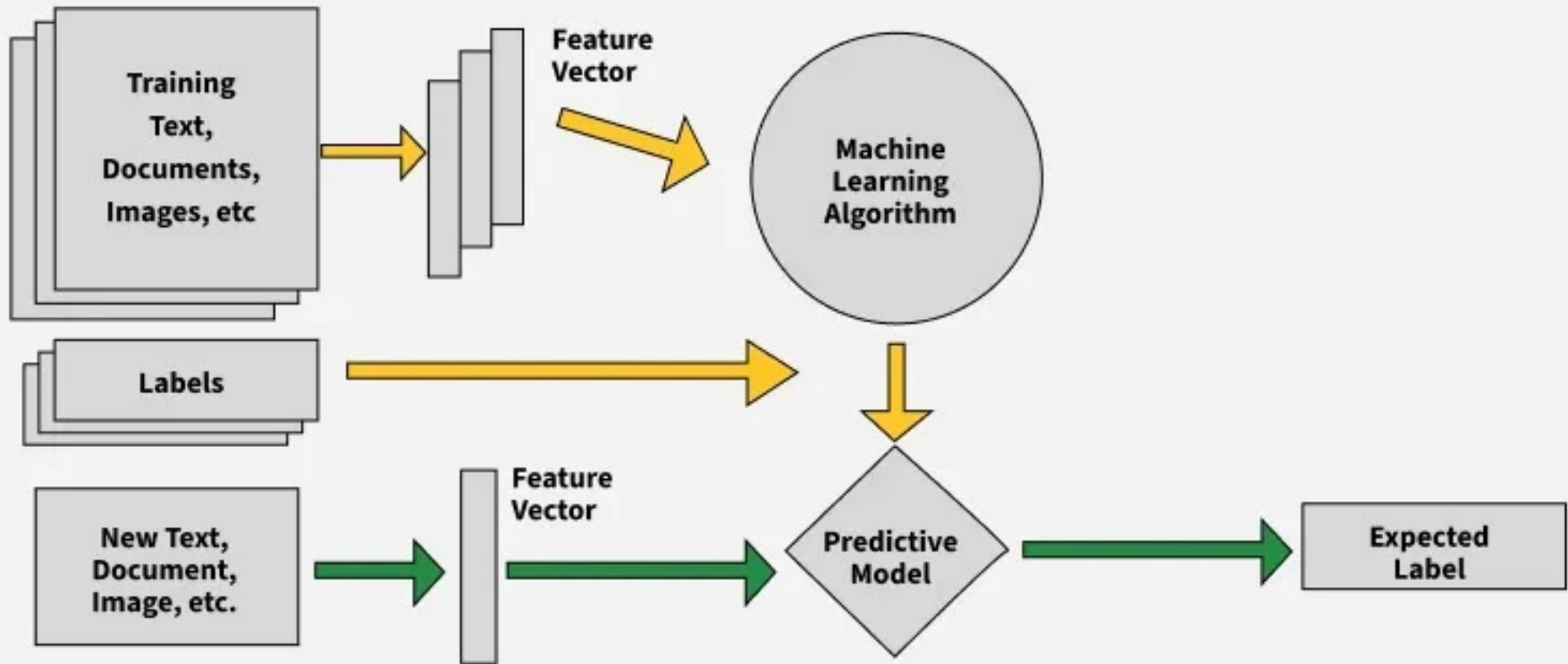
$$P(\text{Class}|\text{Features}) = \frac{P(\text{Features}|\text{Class}) \cdot P(\text{Class})}{P(\text{Features})}$$

Trong đó:

- $P(\text{Class}|\text{Features})$ (**Xác suất Hậu nghiệm**): Xác suất một tài liệu thuộc về một lớp cụ thể, khi biết các đặc trưng (từ) của nó.
- $P(\text{Features}|\text{Class})$ (**Xác suất Khả năng**): Xác suất các đặc trưng xuất hiện, khi biết tài liệu thuộc về một lớp cụ thể.
- $P(\text{Class})$ (**Xác suất Tiên nghiệm**): Xác suất một tài liệu thuộc về một lớp bất kỳ.
- $P(\text{Features})$ (**Xác suất Bằng chứng**): Xác suất các đặc trưng xuất hiện. Đây là hằng số cho tất cả các lớp, nên chúng ta có thể bỏ qua nó nếu so sánh xác suất.

Naive Bayes cho Phân loại Văn bản

Text Classification using naive bayes



Công thức Naive Bayes cho Phân loại Văn bản

- Dựa vào giả định độc lập, công thức Naive Bayes cho phân loại văn bản được đơn giản hóa:

$$P(y|x_1, x_2, \dots, x_n) \propto P(y) \cdot P(x_1|y) \cdot P(x_2|y) \dots P(x_n|y)$$

Trong đó:

y : Class (ví dụ: “spam” hoặc “ham”).

$x_1, x_2, \dots, x_n$: Features - Các đặc trưng của tài liệu

Mục tiêu: Tính toán xác suất này cho mỗi lớp và chọn lớp có xác suất cao nhất: $\hat{y} = \operatorname{argmax}_y (P(y) \prod_{i=1}^n P(x_i|y))$

Ưu nhược điểm của Naive Bayes

Ưu điểm

- Đơn giản, dễ cài đặt.
- Huấn luyện & dự đoán nhanh.
- Hoạt động tốt với dữ liệu ít.
- Xử lý tốt dữ liệu nhiều chiều (văn bản có từ vựng lớn).
- Kết quả dễ giải thích.

Nhược điểm

- Giả định độc lập giữa các từ (không thực tế).
- Bỏ qua ngữ cảnh và vị trí từ.
- Khó xử lý từ hiếm, từ mới.
- Nhạy cảm với dữ liệu mất cân bằng.
- Độ chính xác thấp hơn Logistic Regression, SVM, Deep Learning.

Naive Bayes: Phân loại thư rác

Bài toán: Phân loại một email mới có nội dung "buy cheap shoes" là **thư rác (spam)** hay không (**ham**)

Dữ liệu huấn luyện:

- **Lớp Spam:** 3 tài liệu
 - "buy viagra now"
 - "free viagra"
 - "free online"
- **Lớp Ham:** 2 tài liệu
 - "buy shoes"
 - "free pizza"

Naive Bayes: Phân loại thư rác

Bước 1: Tính xác suất tiên nghiệm

$$P(\text{spam}) = \frac{\text{số tài liệu spam}}{\text{tổng số tài liệu}} = \frac{3}{5} = 0.6$$

$$P(\text{ham}) = \frac{\text{số tài liệu ham}}{\text{tổng số tài liệu}} = \frac{2}{5} = 0.4$$

Bước 2: Tính xác suất khả năng $P(x_i|y)$

- Tính xác suất của mỗi từ ("buy", "cheap", "shoes") trong mỗi lớp.
- Để tránh xác suất bằng 0 cho các từ không xuất hiện, chúng ta sử dụng

Làm mịn Laplace (Laplace Smoothing):

$$P(x_i|y) = \frac{\text{số lần từ } i \text{ xuất hiện trong lớp } y + 1}{\text{tổng số từ trong lớp } y + \text{số từ vựng}}$$

Tính toán cho lớp Spam:

- Tổng số từ: "buy", "viagra", "now", "free", "viagra", "free", "online" -> 7 từ.
- Xây vocabulary (từ duy nhất từ tập huấn luyện): { buy, viagra, now, free, online, shoes, pizza } → $V = 7$

Naive Bayes: Phân loại thư rác

$$P(buy|spam) = \frac{1+1}{7+7} = \frac{2}{14} = 0.143$$

$$P(cheap|spam) = \frac{0+1}{7+7} = \frac{1}{14} = 0.071$$

$$P(shoes|spam) = \frac{0+1}{7+7} = \frac{1}{14} = 0.071$$

Tính toán cho lớp Ham:

- Tổng số từ: "buy", "shoes", "free", "pizza" -> 4 từ.

$$P(buy|ham) = \frac{1+1}{4+7} = \frac{2}{11} = 0.182$$

$$P(cheap|ham) = \frac{0+1}{4+7} = \frac{1}{11} = 0.091$$

$$P(shoes|ham) = \frac{1+1}{4+7} = \frac{2}{11} = 0.182$$

Naive Bayes: Phân loại thư rác

Bước 3: So sánh xác suất và đưa ra kết quả

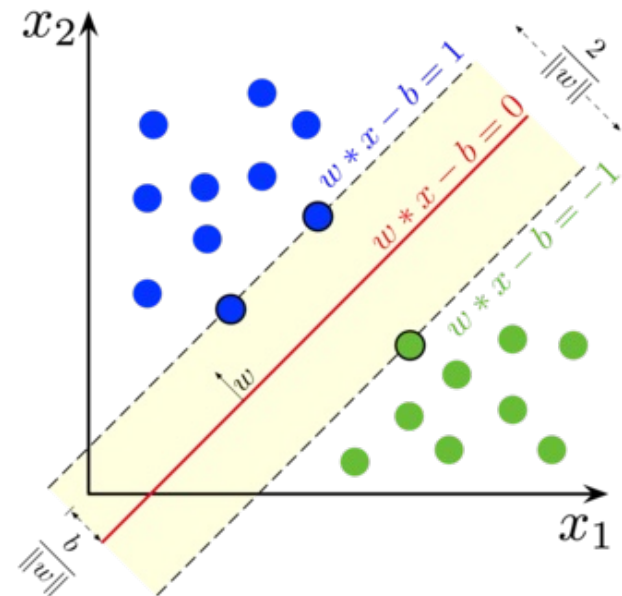
$$\begin{aligned} P1 &= P(\text{spam} | \text{"buy cheap shoes"}) \\ &\propto P(\text{spam}) \cdot P(\text{buy} | \text{spam}) \cdot P(\text{cheap} | \text{spam}) \cdot P(\text{shoes} | \text{spam}) \\ &= \frac{3}{5} \cdot \frac{2}{14} \cdot \frac{1}{14} \cdot \frac{1}{14} = \mathbf{0.00044} \end{aligned}$$

$$\begin{aligned} P2 &= P(\text{ham} | \text{"buy cheap shoes"}) \\ &\propto P(\text{ham}) \cdot P(\text{buy} | \text{ham}) \cdot P(\text{cheap} | \text{ham}) \cdot P(\text{shoes} | \text{ham}) \\ &= \frac{2}{5} \cdot \frac{2}{11} \cdot \frac{1}{11} \cdot \frac{2}{11} = \mathbf{0.00120} \end{aligned}$$

Vì $P2 > P1$, nên lớp được chọn là "ham"

Support Vector Machine (SVM)

- **Support Vector Machine (SVM)** là một thuật toán học có giám sát mạnh mẽ được sử dụng để phân loại và hồi quy.
- **Nguyên tắc cốt lõi:** Tìm một **siêu mặt phẳng (hyperplane)** tối ưu nhất để phân tách các lớp dữ liệu.
- SVM đặc biệt hiệu quả với các bài toán phân loại nhị phân (binary classification) và có thể mở rộng cho phân loại đa lớp (multi-class classification).
- Trong phân loại văn bản, SVM hoạt động trên các vector văn bản (như TF-IDF hoặc các vector Word2Vec) để phân chia chúng thành các lớp khác nhau (ví dụ: spam và ham).



Support Vector Machine (SVM)

- **Siêu mặt phẳng (Hyperplane):** Đây là ranh giới quyết định. Nó là một mặt phẳng trong không gian N chiều (N là số đặc trưng) phân chia các điểm dữ liệu.
- **Vector hỗ trợ (Support Vectors):** Đây là các điểm dữ liệu gần siêu mặt phẳng nhất. Chúng là những điểm quan trọng nhất trong việc định nghĩa siêu mặt phẳng và lề (margin).
- **Lề (Margin):** Khoảng cách giữa siêu mặt phẳng và các vector hỗ trợ gần nhất của mỗi lớp. SVM tìm cách **tối đa hóa lề** này để đạt được sự phân tách tốt nhất và khái quát hóa tốt hơn.

Support Vector Machine (SVM)

Trong không gian n chiều, siêu phẳng được biểu diễn như sau:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

Trong đó:

- $\mathbf{w} = [w_1, w_2, \dots, w_n]$ là **vector trọng số** (hướng pháp tuyến của siêu phẳng).
- $\mathbf{x} = [x_1, x_2, \dots, x_n]$ là **vector dữ liệu**.
- b là **bias** (hoặc intercept), quyết định siêu phẳng dịch chuyển ra xa gốc tọa độ.

SVM cố gắng **tối đa hóa margin**, tương đương với:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

với ràng buộc:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad \forall i$$

- $y_i \in \{+1, -1\}$ là nhãn lớp.

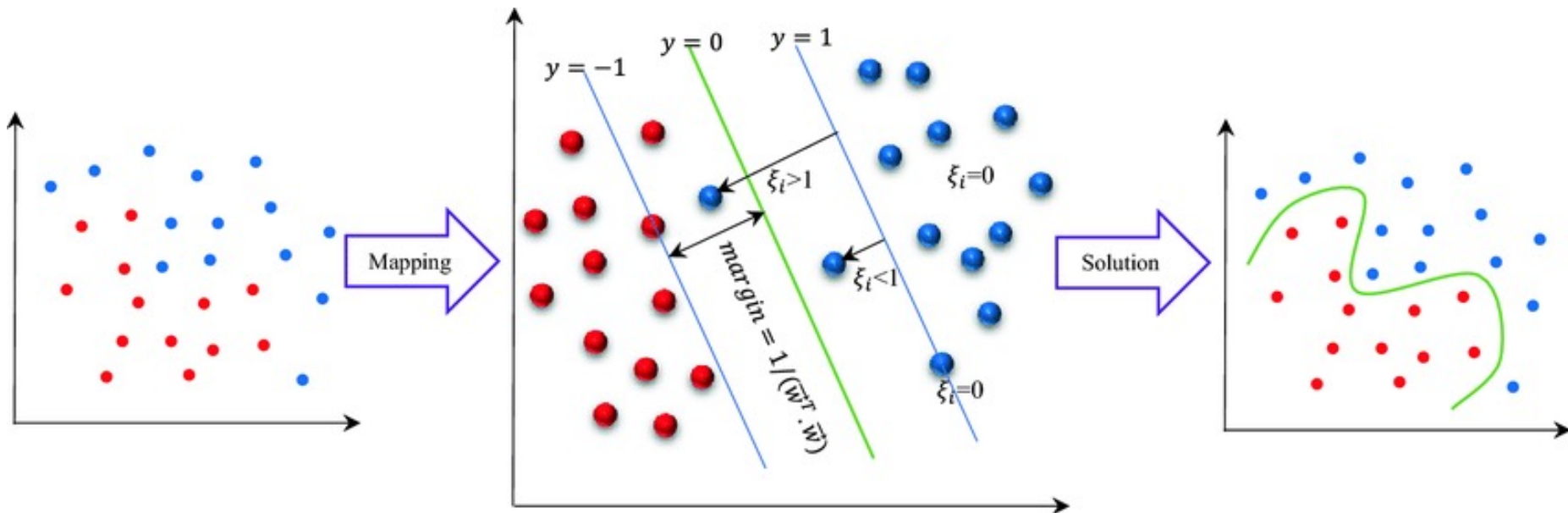
Support Vector Classifier (SVC)

Support Vector Classifier (SVC): mô hình phân loại nhị phân.

Tìm **siêu phẳng (hyperplane)** tối ưu để tách 2 lớp.

Chỉ các điểm gần đường biên (support vectors) ảnh hưởng đến mô hình.

Mục tiêu: **tối đa hóa margin** giữa các lớp.



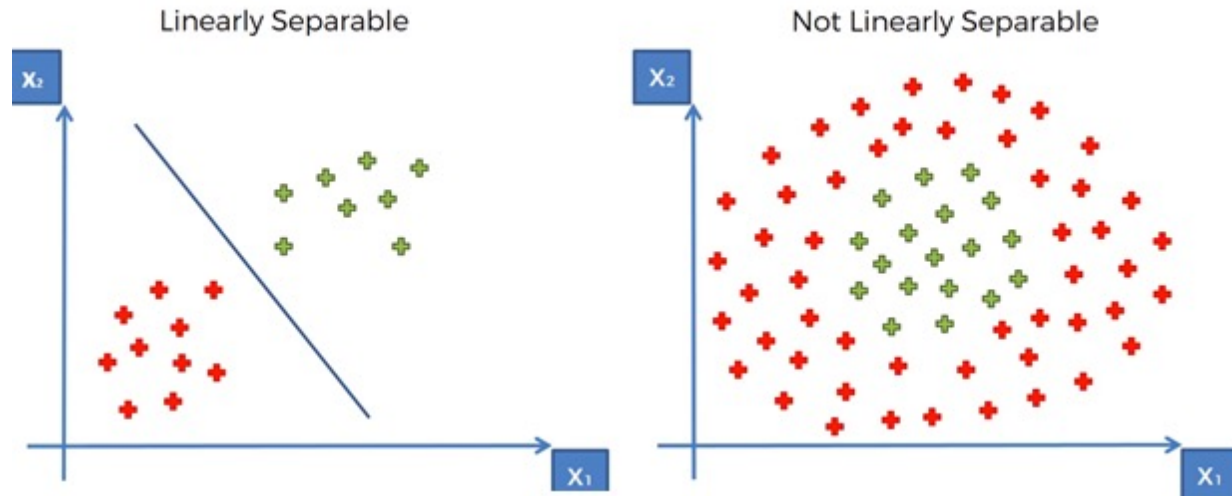
Từ SVC đến SVM

SVC là trường hợp phân loại nhị phân trong SVM.

SVM: mở rộng ý tưởng SVC cho:

- **Dữ liệu phi tuyến** (dùng *kernel trick*).
- **Nhiều lớp** (multi-class classification).

Ứng dụng rộng rãi trong **phân loại văn bản**, xử lý dữ liệu high-dimensional.



Phân tách tuyến tính và phi tuyến tính

SVM tuyến tính (Linear SVM):

- Hoạt động khi dữ liệu có thể được phân tách rõ ràng bằng một đường thẳng hoặc một mặt phẳng.
- Trong phân loại văn bản, điều này có nghĩa là các vector từ của các lớp khác nhau có thể được phân tách bằng một siêu mặt phẳng.

SVM phi tuyến tính (Non-linear SVM):

- Khi dữ liệu không thể phân tách bằng một đường thẳng.
- SVM sử dụng "**hàm nhân**" (**kernel function**) để ánh xạ dữ liệu ban đầu lên một không gian có chiều cao hơn, nơi dữ liệu trở nên có thể phân tách tuyến tính.

SVM trong phân loại văn bản

Bước 1: Vector hóa văn bản:

- Biến văn bản thành vector số bằng các kỹ thuật như **Bag of Words**, **TF-IDF**, hoặc **Word2Vec**. TF-IDF là lựa chọn rất phổ biến và hiệu quả khi kết hợp với SVM.

Bước 2: Huấn luyện mô hình:

- Dùng tập dữ liệu đã được vector hóa để huấn luyện mô hình SVM. Mô hình sẽ tìm ra siêu mặt phẳng tối ưu nhất để phân loại các vector văn bản này.

Bước 3: Dự đoán:

- Với một tài liệu mới, chúng ta vector hóa nó bằng cùng một phương pháp đã dùng để huấn luyện.
- Sau đó, mô hình SVM sẽ sử dụng siêu mặt phẳng đã học để dự đoán lớp của tài liệu mới đó.

Ưu nhược điểm của SVM

Ưu điểm:

- **Hiệu quả cao trong không gian đa chiều:** SVM hoạt động rất tốt với các bộ dữ liệu có nhiều đặc trưng hơn số lượng mẫu, điều thường thấy trong phân loại văn bản.
- **Khả năng khái quát hóa tốt:** Nhờ việc tối đa hóa lề, SVM có xu hướng tránh được việc học quá khớp (overfitting).
- **Đa năng:** Có thể sử dụng các hàm nhân khác nhau để xử lý các bài toán phân loại phi tuyến tính.

Nhược điểm:

- **Tốn kém về mặt tính toán:** Việc huấn luyện SVM có thể tốn kém tài nguyên và chậm trên các tập dữ liệu rất lớn.
- **Khó giải thích:** Kết quả của SVM không trực quan như các mô hình dựa trên cây quyết định. Rất khó để giải thích tại sao một điểm dữ liệu cụ thể lại được phân loại như vậy.

Đánh giá các mô hình phân loại - Tại sao cần đánh giá?

- **Mục tiêu:** Xác định mức độ tốt của mô hình trong việc dự đoán các nhãn mới, chưa từng thấy.
- **Tránh overfitting:** Đảm bảo mô hình không chỉ học thuộc lòng dữ liệu huấn luyện mà còn có khả năng tổng quát hóa.
- **So sánh mô hình:** Lựa chọn mô hình tốt nhất cho bài toán cụ thể.
- **Đảm bảo độ tin cậy:** Cung cấp số liệu định lượng về hiệu suất của mô hình.

- **Tập huấn luyện (Training Set):** Phần lớn dữ liệu được sử dụng để huấn luyện mô hình (thường 70-80%).
- **Tập kiểm tra (Test Set):** Phần còn lại của dữ liệu, hoàn toàn riêng biệt, được sử dụng để đánh giá hiệu suất của mô hình sau khi huấn luyện.
- **Tập kiểm định (Validation Set):** Đôi khi được sử dụng để tinh chỉnh tham số mô hình trong quá trình huấn luyện, trước khi đánh giá cuối cùng trên tập kiểm tra.

Ma trận nhầm lẫn (Confusion Matrix)

Confusion Matrix là bảng tóm tắt hiệu suất của một thuật toán phân loại, cho thấy số lượng các dự đoán đúng và sai của mô hình so với nhãn thực tế.

Các thành phần:

- **True Positive (TP):** Thực tế là Positive, dự đoán là Positive.
- **True Negative (TN):** Thực tế là Negative, dự đoán là Negative.
- **False Positive (FP):** Thực tế là Negative, dự đoán là Positive (Lỗi loại I).
- **False Negative (FN):** Thực tế là Positive, dự đoán là Negative (Lỗi loại II).

Confusion matrix for binary classification

Actual value	A	TP	FN
	B	FP	TN
		A	B
		Predicted value	

Các chỉ số đánh giá

Accuracy:

$$Accuracy = \frac{TP + TN}{FP + FN + TP + TN}$$

- **Ý nghĩa:** Tỷ lệ các dự đoán đúng trên tổng số dự đoán.
- **Hạn chế:** Không phù hợp khi dữ liệu không cân bằng (imbalanced data).

Precision:

$$Precision = \frac{TP}{TP + FP}$$

- **Ý nghĩa:** Trong số các mẫu được dự đoán là Positive, có bao nhiêu mẫu thực sự là Positive.
- **Ưu tiên:** Khi chi phí của FP là cao (ví dụ: hệ thống spam email, không muốn đánh dấu nhầm email quan trọng là spam).

Các chỉ số đánh giá

Độ phủ (Recall/Sensitivity):

$$\text{Recall} = \frac{TP}{TP + FN}$$

Ý nghĩa: Trong số các mẫu thực sự là Positive, mô hình đã tìm thấy bao nhiêu mẫu.

- **Ưu tiên:** Khi chi phí của FN là cao (ví dụ: phát hiện bệnh, không muốn bỏ sót người bệnh).

F1-Score:

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Ý nghĩa: Giá trị trung bình điều hòa của Precision và Recall. Cân bằng giữa hai chỉ số này, đặc biệt hữu ích khi có sự mất cân bằng giữa Precision và Recall.

Sử dụng ma trận nhầm lẫn đánh giá các chỉ số đã học.

Thực tế / Dự đoán	Lớp 1 (Positive)	Lớp 0 (Negative)
Lớp 1 (Positive)	TP = 90	FN = 10
Lớp 0 (Negative)	FP = 5	TN = 95

- **Accuracy:** $(90+95)/(90+95+5+10)=185/200=0.925$ (92.5%)
- **Precision:** $90/(90+5)=90/95\approx 0.947$ (94.7%)
- **Recall:** $90/(90+10)=90/100=0.9$ (90%)
- **F1-Score:** $2\times(0.947\times 0.9)/(0.947+0.9)\approx 0.923$ (92.3%)

Đường cong ROC và AUC

ROC Curve (Receiver Operating Characteristic Curve):

- Vẽ mối quan hệ giữa True Positive Rate (TPR = Recall) và False Positive Rate (FPR = FP / (FP + TN)) ở các ngưỡng phân loại khác nhau.
- Mô hình tốt sẽ có đường cong gần góc trên bên trái của biểu đồ.

AUC (Area Under the Curve):

- Diện tích dưới đường cong ROC.
- Giá trị từ 0 đến 1. AUC càng gần 1, mô hình càng tốt trong việc phân loại.
- AUC = 0.5 nghĩa là mô hình dự đoán ngẫu nhiên.

